

Pickle Rick

Penetration Testing Report

Authorized TryHackMe Lab Assessment

Assessment type: Black-box simulated penetration test
Environment: TryHackMe - intentionally vulnerable training machine
Assessment date: September 18, 2026

Cybersecurity Portfolio Report

1. Executive Summary

A security assessment was conducted against the intentionally vulnerable Pickle Rick TryHackMe machine. The objective was to identify weaknesses that could permit unauthorized access, operating-system command execution, and privilege escalation within the authorized lab environment.

Reconnaissance identified SSH and HTTP services. Web enumeration exposed authentication-related information through publicly accessible resources. After authenticating to the application, a command panel allowed operating-system commands to execute as the web-service account **www-data**. Local privilege enumeration then revealed unrestricted passwordless sudo privileges, which were validated by accessing the root user's directory with elevated privileges.

The assessment demonstrates how information disclosure, dangerous application functionality, and excessive local privileges can be chained to produce complete host compromise.

2. Scope and Rules of Engagement

Item	Details
Target	Pickle Rick - TryHackMe
Environment	Authorized, intentionally vulnerable training environment
Assessment approach	Black-box reconnaissance and exploitation
Objective	Identify and validate weaknesses leading to unauthorized access and privilege escalation
Out of scope	Systems outside the assigned TryHackMe target

The temporary lab IP address visible in screenshots is retained only as evidence of the authorized assessment session and is not a persistent production asset.

3. Methodology

The assessment followed a structured workflow: reconnaissance, service enumeration, web enumeration, authentication testing, command-execution validation, local enumeration, privilege analysis, and impact validation.

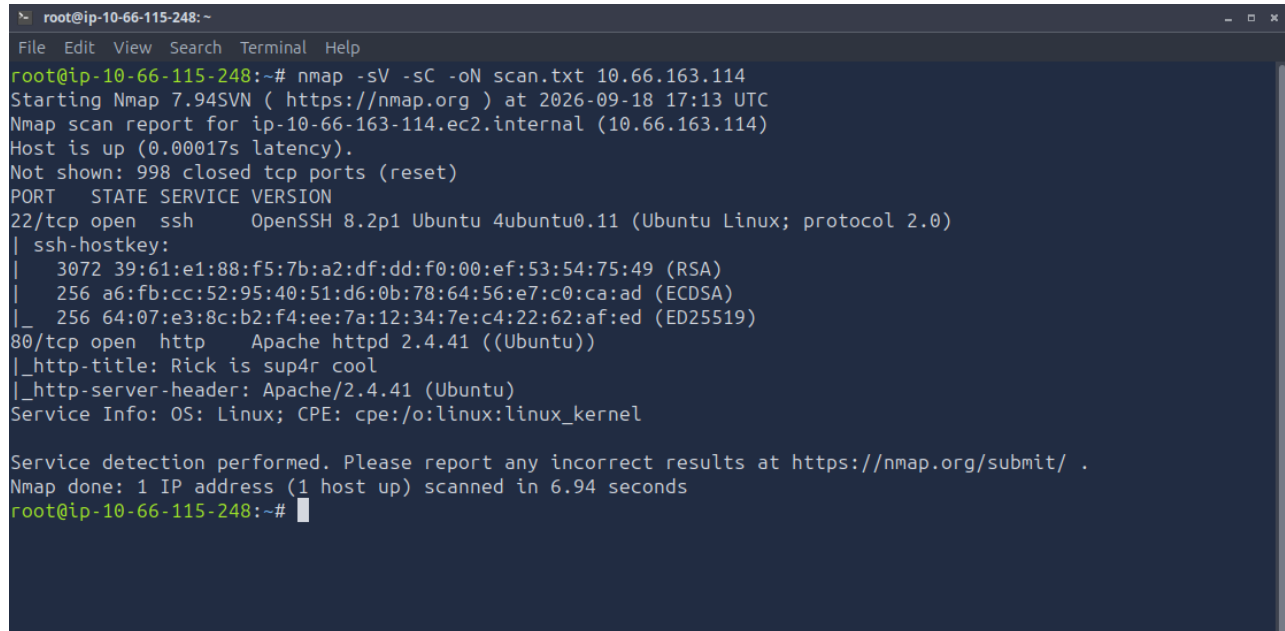
Phase	Purpose
Reconnaissance	Identify reachable services and technologies.
Web enumeration	Discover application endpoints and exposed resources.
Initial access	Determine whether discovered information permits authenticated access.
Execution validation	Establish whether the application can execute operating-system commands.
Local enumeration	Determine the permissions and accessible resources of the compromised account.
Privilege escalation	Identify excessive privileges and validate resulting impact.

4. Reconnaissance and Enumeration

An Nmap service and default-script scan was performed to identify exposed TCP services and obtain basic version information.

```
nmap -sV -sC -oN scan.txt <TARGET_IP>
```

The scan identified TCP/22 running OpenSSH 8.2p1 and TCP/80 running Apache HTTP Server 2.4.41. Because HTTP presented the primary application attack surface, subsequent enumeration focused on the web server.



```
root@ip-10-66-115-248: ~
File Edit View Search Terminal Help
root@ip-10-66-115-248:~# nmap -sV -sC -oN scan.txt 10.66.163.114
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-18 17:13 UTC
Nmap scan report for ip-10-66-163-114.ec2.internal (10.66.163.114)
Host is up (0.00017s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.11 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 39:61:e1:88:f5:7b:a2:df:dd:f0:00:ef:53:54:75:49 (RSA)
|   256  a6:fb:cc:52:95:40:51:d6:0b:78:64:56:e7:c0:ca:ad (ECDSA)
|_  256  64:07:e3:8c:b2:f4:ee:7a:12:34:7e:c4:22:62:af:ed (ED25519)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
|_ _http-title: Rick is sup4r cool
|_ _http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.94 seconds
root@ip-10-66-115-248:~#
```

Figure 1. Initial Nmap scan identifying SSH and HTTP services.

4.1 Web Content Enumeration

Gobuster was used to identify common directories and files, including PHP, HTML, and text resources.

```
gobuster dir -u http://<TARGET_IP>/ -w /usr/share/wordlists/dirb/common.txt -x php,html,txt
```

Enumeration identified noteworthy resources including **/login.php**, **/portal.php**, and **/robots.txt**. These resources were manually inspected for information relevant to authentication and application access.

```
root@ip-10-66-115-248:~  
File Edit View Search Terminal Help  
root@ip-10-66-115-248:~# gobuster dir -u http://10.66.163.114/ -w /usr/share/wordlists/dirb/common.txt -x php,html,txt  
=====
```

gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehrlauer (@firefart)

```
=====
```

+] Url: http://10.66.163.114/
+] Method: GET
+] Threads: 10
+] Wordlist: /usr/share/wordlists/dirb/common.txt
+] Negative Status codes: 404
+] User Agent: gobuster/3.6
+] Extensions: php,html,txt
+] Timeout: 10s

```
=====
```

Starting gobuster in directory enumeration mode

```
=====
```

.php	(Status: 403)	[Size: 278]
.hta	(Status: 200)	[Size: 1062]
.htaccess	(Status: 403)	[Size: 278]
.hta	(Status: 403)	[Size: 278]
.htaccess.php	(Status: 403)	[Size: 278]
.htpasswd.php	(Status: 403)	[Size: 278]
.hta.php	(Status: 403)	[Size: 278]
.htpasswd	(Status: 403)	[Size: 278]
.htpasswd	(Status: 403)	[Size: 278]
.htaccess	(Status: 403)	[Size: 278]
assets	(Status: 301)	[Size: 315]
denied.php	(Status: 302)	[Size: 0]
index.html	(Status: 200)	[Size: 1062]
login.php	(Status: 200)	[Size: 882]
portal.php	(Status: 302)	[Size: 0]
robots.txt	(Status: 200)	[Size: 17]
server-status	(Status: 403)	[Size: 278]

```
Progress: 13842 / 13845 (99.98%)  
=====
```

Finished

```
=====
```

root@ip-10-66-115-248:~#

Figure 2. Gobuster results revealing authentication and application endpoints.

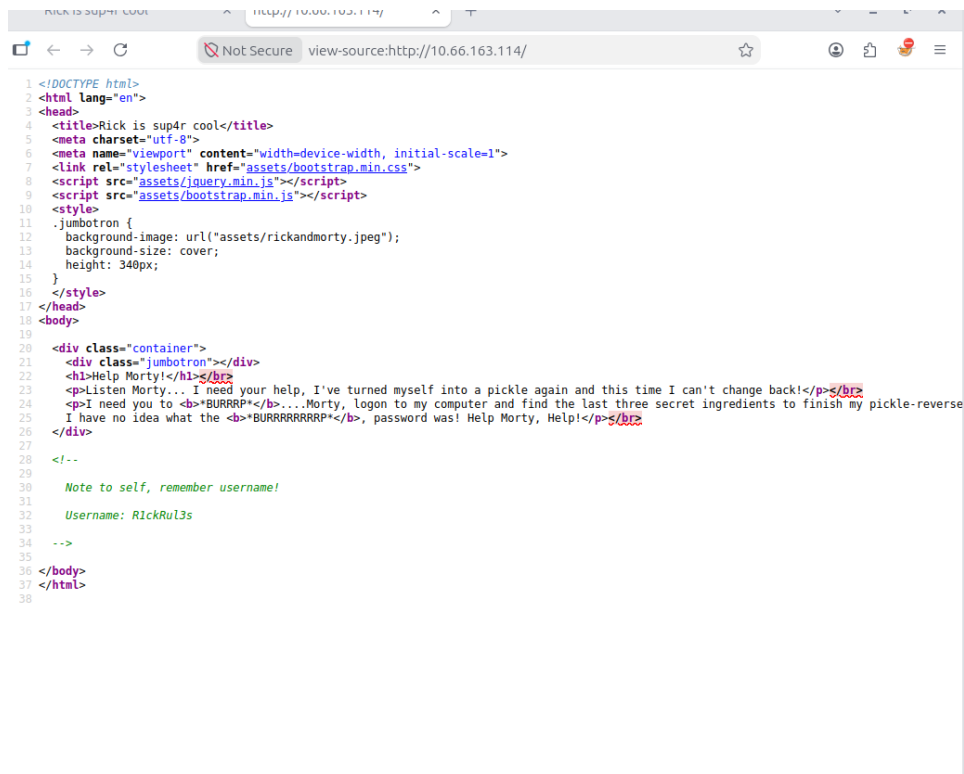
5. Findings

The findings below represent weaknesses that materially contributed to the attack path. Severity labels are qualitative portfolio assessments based on demonstrated lab impact rather than formal CVSS scores.

Finding 1 - Authentication Information Exposed in Public Web Content

Severity: High

Description. Manual inspection of the publicly accessible HTML source revealed a username embedded in an HTML comment. A separate public **robots.txt** resource exposed an additional string that was usable as authentication information. Combining these disclosures provided the information needed to access the protected portal.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <title>Rick is sup4r cool</title>
5 <meta charset="utf-8">
6 <meta name="viewport" content="width=device-width, initial-scale=1">
7 <link rel="stylesheet" href="assets/bootstrap.min.css">
8 <script src="assets/jquery.min.js"></script>
9 <script src="assets/bootstrap.min.js"></script>
10 <style>
11 .jumbotron {
12   background-image: url("assets/rickandmorty.jpeg");
13   background-size: cover;
14   height: 340px;
15 }
16 </style>
17 </head>
18 <body>
19
20 <div class="container">
21 <div class="jumbotron"></div>
22 <h1>Help Morty!</h1></div>
23 <p>Listen Morty... I need your help, I've turned myself into a pickle again and this time I can't change back!</p></div>
24 <p>I need you to <b>BURRRRP</b>...Morty, logon to my computer and find the last three secret ingredients to finish my pickle-reverse
25 I have no idea what the <b>BURRRRRRRRP</b>, password was! Help Morty, Help!</p></div>
26
27 <!--
28
29   Note to self, remember username!
30
31   Username: RickRu13s
32
33 -->
34
35
36 </body>
37 </html>
38
```

Figure 3. Username disclosed in publicly accessible page source.

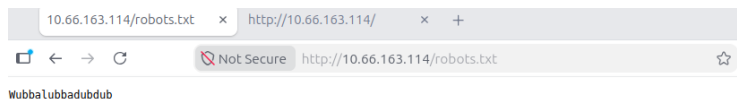


Figure 4. Additional authentication information exposed through robots.txt.

Impact. An unauthenticated user could retrieve authentication-related information without exploiting a software vulnerability. Successful use of the exposed information provided access to sensitive application functionality.

Remediation. Do not store credentials, secrets, or authentication hints in client-accessible source code or public files. Remove secrets from the web root, rotate exposed credentials, and use secure secret-management practices.

Finding 2 - Authenticated Operating-System Command Execution

Severity: Critical

Description. After successful authentication, the application exposed a command panel that accepted operating-system commands and returned their output. This functionality effectively provided arbitrary command execution through the web application.

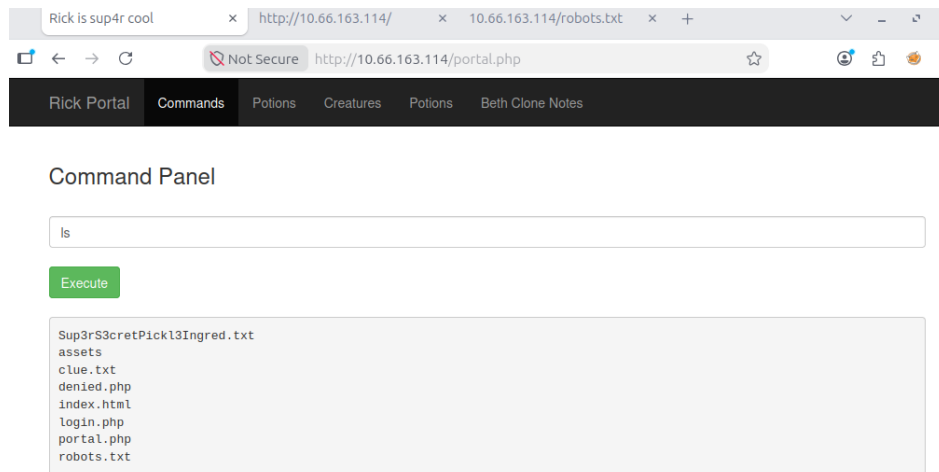


Figure 5. Authenticated command panel exposed by the application.

The execution context was verified using **whoami**, which showed that commands were running as the Apache web-service account **www-data**.

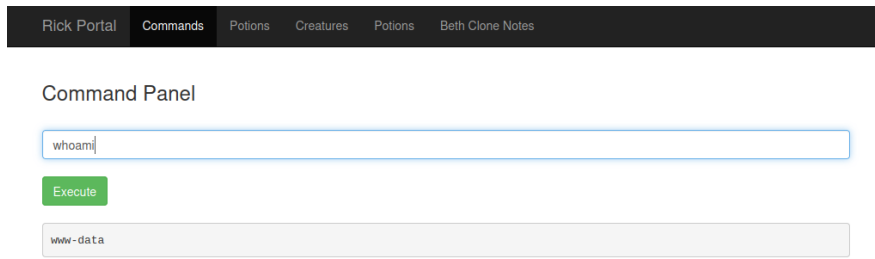


Figure 6. Command execution confirmed under the www-data service account.

Impact. A user who obtains portal access can execute commands on the underlying Linux host. This converts an application-level compromise into operating-system access and creates a path toward sensitive-file access and privilege escalation.

Remediation. Remove general-purpose operating-system command execution from the application. Where backend command execution is unavoidable, use strict allowlists, fixed arguments, strong authorization, input validation, and process isolation. The web-service account should operate with minimal privileges.

6. Local Enumeration

Following command-execution validation, the local filesystem was enumerated to identify user data accessible from the compromised web-service context. The **/home/rick** directory contained a file owned by root that was visible from the application context.

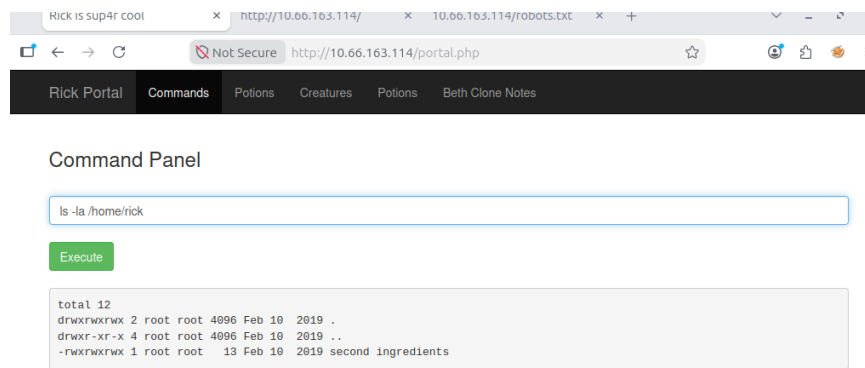


Figure 7. Local enumeration of the Rick user's home directory.

Finding 3 - Unrestricted Passwordless Sudo for the Web-Service Account

Severity: Critical

Description. Local privilege enumeration with **sudo -l** showed that **www-data** could run all commands as any user without supplying a password: **(ALL) NOPASSWD: ALL**. A web-service account should not possess unrestricted administrative privileges.



Figure 8. sudo -l output showing unrestricted passwordless sudo privileges for www-data.

Impact. Any attacker who achieves command execution as **www-data** can immediately execute commands with root privileges, resulting in complete compromise of the host.

Remediation. Remove **NOPASSWD: ALL** from the web-service account. Apply least privilege and, if sudo access is genuinely required, restrict it to narrowly defined commands and arguments that cannot be abused for arbitrary code execution.

7. Privilege Escalation and Impact Validation

The sudo misconfiguration was validated by using elevated privileges to list the root user's directory. Successful access confirmed that the application compromise could be escalated from **www-data** to effective root-level access.

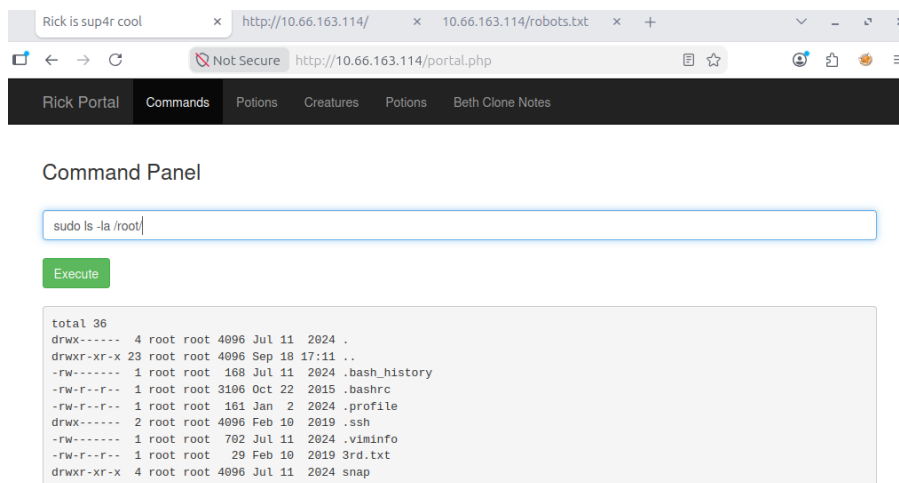


Figure 9. Successful privileged access to the root user's directory.

The assessment stopped after demonstrating the security impact required by the lab objectives. No persistence or destructive actions were necessary.

8. Attack Path Summary

Stage	Observed result
1. Network reconnaissance	SSH and HTTP services identified.
2. Web enumeration	Login, portal, and robots.txt resources discovered.
3. Information disclosure	Authentication-related information exposed in public web content.
4. Authenticated access	Protected portal became accessible.
5. Command execution	Operating-system commands executed as www-data.
6. Privilege enumeration	www-data found to have (ALL) NOPASSWD: ALL.
7. Impact validation	Privileged access to /root successfully demonstrated.

9. Remediation Summary

Finding	Priority	Recommended action
Authentication information exposed in public web content	High	Remove secrets and authentication information from client-accessible files; rotate
Authenticated operating-system command execution	Critical	Remove arbitrary command functionality or tightly constrain it using allowlists, a
Unrestricted passwordless sudo for www-data	Critical	Remove NOPASSWD: ALL and apply least privilege to the web-service account

10. Conclusion

The Pickle Rick assessment demonstrated a complete attack chain from external reconnaissance to privileged host access. The HTTP service exposed information that assisted authentication, the authenticated portal provided direct operating-system command execution, and the impact became critical because the web-service account possessed unrestricted passwordless sudo privileges.

The central lesson is that weaknesses should not be evaluated only in isolation. An exposed username, a public text file, dangerous application functionality, and an excessive sudo rule were separate configuration problems; chained together, they enabled full compromise of the target.

11. Skills Demonstrated

Network reconnaissance, Nmap service enumeration, web content discovery with Gobuster, manual source inspection, authentication testing, Linux command execution, filesystem enumeration, sudo privilege analysis, privilege-escalation validation, evidence collection, and security-report writing.

Lab disclaimer: All testing documented in this report was performed against an intentionally vulnerable TryHackMe training machine in an authorized environment. This report is intended as a cybersecurity portfolio artifact.